

RAPPORT DE STAGE

Première année — BTS SIO option SLAM

Agence Malagasy de Presse (AMP)

Effectué par :

ANDRIANAMBININTSOA Benja Manjaka Ginola

Période :

Du 02 juin au 12 juillet 2025

Encadrement :

Tuteur en entreprise : Monsieur FRANCIS RASOAMAHARO

Tuteur académique : Monsieur Nasser MAUDARBOCUS

MCCI BUSINESS SCHOOL — Cyber Cité d'Ebène, Île Maurice

AGENCE MALAGASY DE PRESSE — Besarety, Antananarivo, Madagascar

Année universitaire 2024 — 2025

SOMMAIRE

Remerciements	3
Introduction	4
I. L'entreprise et mon poste	5
1. L'Agence Malagasy de Presse	5
2. Mon rôle de stagiaire SIO	5
II. WordPress	6
1. Découverte de WordPress	6
2. Le tableau de bord	7
3. Le workflow d'un article	7
III. Côté technique : ce que j'ai découvert	8
1. WordPress, MySQL et PHP : qui fait quoi	8
2. Le code que j'ai lu et compris	9
IV. Mes deux missions de développement	11
1. Mission 1 : widget des derniers articles	11
2. Mission 2 : compteur de vues par article	13
V. Les incidents serveur	15
VI. Bilan : ce stage et le BTS SIO SLAM	16
Conclusion	18

REMERCIEMENTS

Je voudrais d'abord remercier Monsieur FRANCIS RASOAMAHARO, mon tuteur à l'AMP, pour sa patience. Il a vraiment pris le temps de m'expliquer un environnement que je ne connaissais pas du tout au début, et sans lui j'aurais mis beaucoup plus de temps à m'y retrouver.

Je remercie aussi Monsieur Nasser MAUDARBOCUS, mon tuteur académique à la MCCI Business School, pour le suivi pendant le stage.

Merci aussi à toute l'équipe de l'AMP. C'était mon premier vrai stage, et j'ai pu poser mes questions sans se retenir

Et merci à l'équipe pédagogique de la MCCI Business School pour les cours de première année, qui m'ont donné les bases dont j'avais besoin pour ce stage.

INTRODUCTION

Ce rapport porte sur mon stage de première année de BTS SIO option SLAM, que j'ai effectué à l'Agence Malagasy de Presse du 2 juin au 12 juillet 2025. Six semaines, j'ai eu le temps de découvrir beaucoup de choses.

L'AMP m'a accueilli dans son équipe technique, et mon rôle était de m'occuper du site web. Concrètement, j'étais là pour suivre le bon fonctionnement de la plateforme, comprendre comment elle marche, et intervenir dessus si besoin.

Avant d'arriver, je n'avais jamais touché à WordPress. Je connaissais le PHP et le SQL grâce aux cours, j'avais déjà fait du HTML et du CSS, c'était nouveau Wordpress pour moi.

Dans la suite, je raconte d'abord l'entreprise et le poste, puis ma découverte de WordPress. Ensuite j'aborde la partie plus technique avec un peu de PHP que j'ai pu regarder. Je présente après les deux missions de développement que mon tuteur m'a confiées vers la fin du stage, et je termine sur les incidents serveur qu'on a eus avant un bilan en lien avec le BTS SIO SLAM.

I. L'ENTREPRISE ET MON POSTE

1. L'Agence Malagasy de Presse

L'Agence Malagasy de Presse, ou AMP, est une agence de presse basée à Antananarivo, à Madagascar. Comme toute agence de ce type, elle joue un rôle de relais : elle collecte de l'information, la traite, puis la diffuse sur son propre site et auprès d'autres médias.

La rédaction dépend du numérique aujourd'hui. Le site web n'est pas un truc en plus, c'est l'outil principal de diffusion. Si le site est en panne, l'agence ne peut plus faire son travail correctement. Ça m'a fait comprendre tout de suite pourquoi mon poste, même de stagiaire, était pris au sérieux.

L'équipe se divise en deux. D'un côté il y a le rédactionnel, avec les journalistes et le directeur de publication, qui produit le contenu. De l'autre côté il y a la technique, où j'étais affecté, qui s'occupe de la mise en ligne et du fonctionnement de la plateforme. Les deux pôles travaillent en permanence ensemble.

2. Mon rôle de stagiaire SIO

Mon poste s'appelait "stagiaire administrateur du site".

Ce qui consiste à :

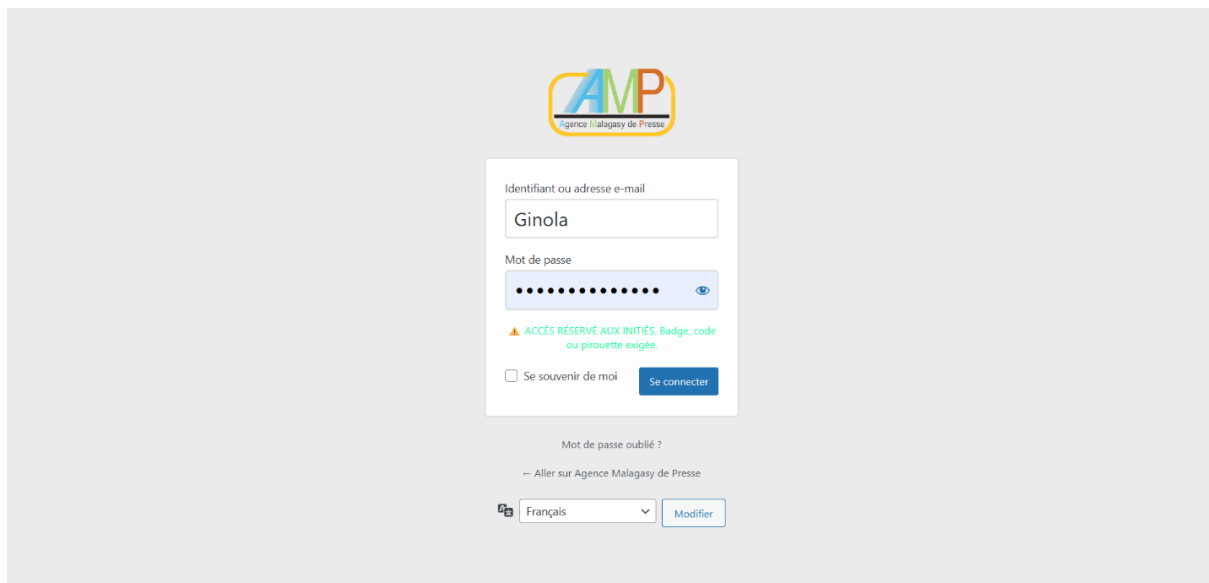
- Vérifier le matin que le site était bien en ligne et qu'il n'y avait pas de soucis visuels ;
- Récupérer les articles validés par la rédaction et les mettre en ligne dans WordPress, en respectant la rubrique demandée ;
- Prévenir mon tuteur quand le site avait un comportement bizarre ;
- Comprendre le fonctionnement du site et surtout WordPress ;
- Exécuté les missions de développement donné par l'équipe technique.

II. WORDPRESS

1. Découverte de Wordpress

Comme je l'ai dit, je n'avais jamais utilisé WordPress avant ce stage. Le premier jour, mon tuteur m'a connecté au tableau de bord du site et m'a expliqué que c'était l'outil que je serais amené à utiliser au quotidien. En découvrant l'interface, avec son menu latéral de nombreuses entrées, ses extensions et ses notifications.

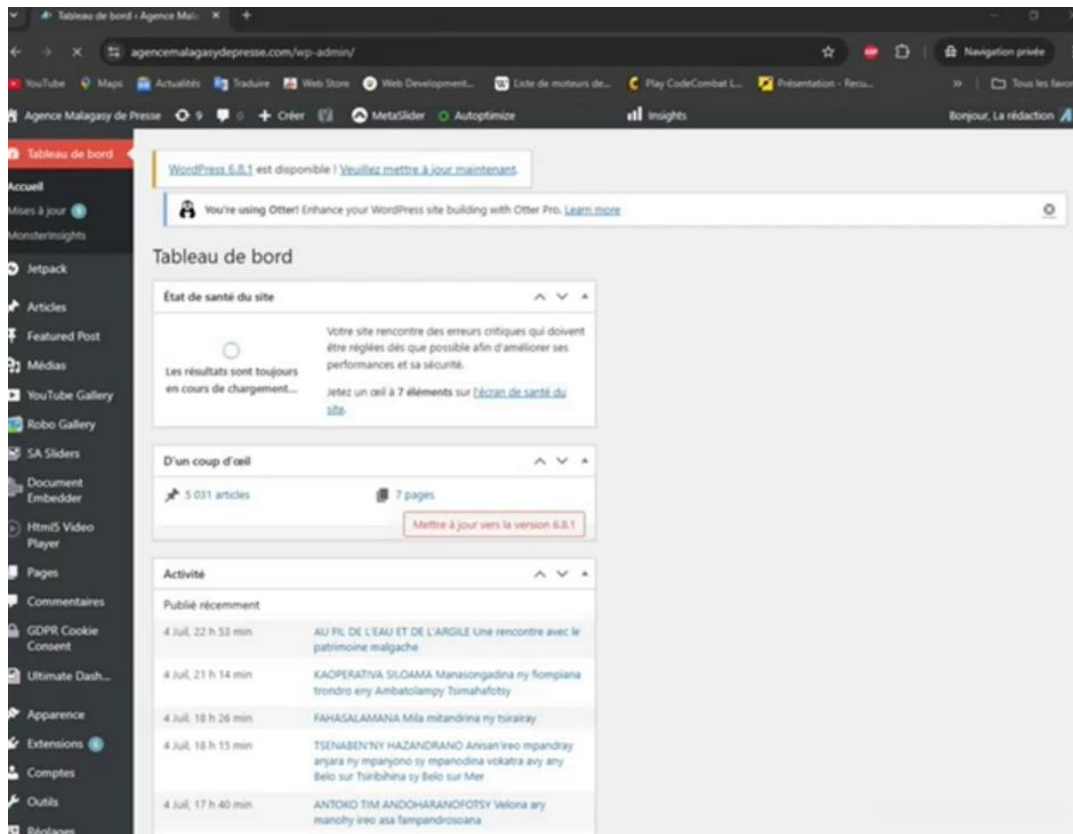
L'élément le plus marquant de ce stage n'a finalement pas été un incident technique en particulier, mais cette obligation d'apprendre WordPress rapidement, en autonomie, dans un contexte professionnel. La capacité à se former vite sur un outil que l'on découvre est, selon moi, le véritable acquis de cette expérience, car elle me sera utile pour tous les outils que je serai amené à manipuler par la suite.



2. Le tableau de bord

J'ai ensuite exploré le tableau de bord plus en détail. Voici les sections que j'ai utilisées le plus :

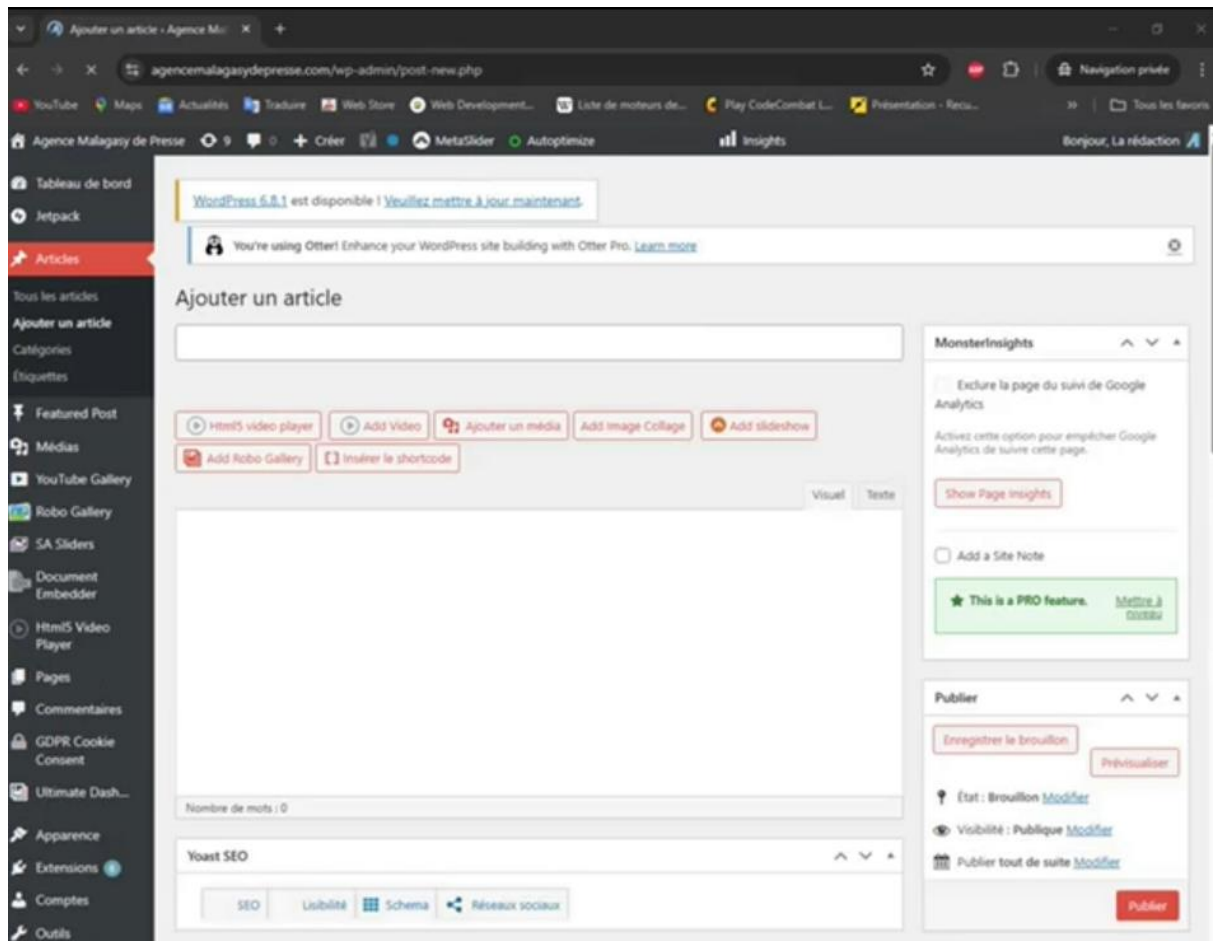
- **Articles** : c'est là que tous les contenus journalistiques sont gérés.
- **Médias** : la bibliothèque des images. Là où l'ensemble des images sont regroupé.
- **Apparence** : le thème, c'est-à-dire les fichiers PHP qui dessinent le site.
- **Extensions** : Les extensions que l'entreprise utilise le plus
- **Utilisateurs** : les comptes et leurs rôles.
- **Réglages** : les paramètres globaux du site.



3. Le workflow d'un article

Même si je n'écrivais pas les articles, je faisais partie de la chaîne de publication, comment une info passe du terrain à l'écran du lecteur. À l'AMP ça se passe en cinq étapes :

- **Rédaction** : le journaliste écrit l'article en dehors de WordPress, souvent sur Word.
- **Validation** : le directeur de publication relit et valide, ou demande des corrections.
- **Mise en ligne** : l'article m'arrive et je l'intègre dans WordPress
- **Contrôle** : l'administrateur principal vérifie que tout s'affiche bien après publication.
- **Consultation** : l'article est en ligne et les lecteurs peuvent le lire.



III. CÔTÉ TECHNIQUE : CE QUE J'AI DÉCOUVERT

1. WordPress, MySQL et PHP : qui fait quoi

Comment WordPress, MySQL et PHP s'articulent en vrai. En cours on étudie ces technos une par une, dans des chapitres séparés. Là, je les voyais tourner ensemble.

Quand un lecteur arrive sur le site, en quelques millisecondes il se passe ça : son navigateur envoie une requête au serveur, Apache passe la requête à PHP, PHP va chercher le contenu dans la base MySQL, assemble une page HTML, et la renvoie au navigateur. WordPress c'est juste un gros programme PHP qui fait ce travail d'assemblage, en suivant des conventions précises.

J'ai aussi pris le temps de regarder la base de données avec phpMyAdmi. Le site de l'AMP, qui paraît si complexe quand on est dessus, repose en fait sur quelques tables bien organisées. Les principales que j'aie retenues sont les suivantes :

NB : Pour des raisons de confidentialité, je ne reproduis pas les noms exacts des tables de la base de l'AMP. Le tableau ci-dessous présent, à titre indicatif, les principales tables d'une base WordPress standard

Table	Ce qu'elle stocke
wp_posts	Tous les articles et pages du site.
wp_users	Les comptes utilisateurs avec leur rôle.
wp_terms	Les catégories et étiquettes des articles.
wp_options	Les paramètres globaux du site.
wp_comments	Les commentaires laissés par les lecteurs.

2. Le code que j'ai lu et compris

Mon tuteur m'a permis de voir quelque code du site, j'ai ouvert plusieurs fichiers PHP sous l'autorisation de mon tuteur du thème pour essayer de comprendre comment tout s'articulait. Je n'ai pas modifié de code en production.

Mon tuteur m'a enseigné quelque code de base essentielles pour le fonctionnement du site.

Voici les 2 principaux que j'ai retenu :

a) La boucle WordPress, ou "The Loop"

C'est un code que l'on retrouve dans presque tous les fichiers du thème WordPress. Son principe est simple : on vérifie s'il y a des articles à afficher, et tant qu'il en reste, on les affiche un par un.

```
<?php
if ( have_posts() ) :
    while ( have_posts() ) : the_post(); ?>

        <h2><?php the_title(); ?></h2>
        <div class="date"><?php the_date(); ?></div>
        <div class="contenu"><?php the_excerpt(); ?></div>

    <?php endwhile;
else : ?>
    <p>Pas d'article à afficher.</p>
```

```
<?php endif; ?>
```

b) Une fonction simple ajoutée via un hook

Mon tuteur m'a expliqué qu'on peut modifier WordPress sans toucher à son code de base, en ajoutant des fonctions dans le fichier `functions.php` du thème. Voici un exemple basique :

```
<?php
function ajouter_signature($contenu) {
    if (is_single()) {
        return $contenu . '<p>@ Agence Malagasy de Presse</p>';
    }
    return $contenu;
}
add_filter('the_content', 'ajouter_signature');
```

Le principe : on demande à WordPress de passer le contenu d'un article par notre fonction avant de l'afficher. Ce mécanisme s'appelle un filtre. Je l'ai réutilisé pour mes deux missions présentées dans la section suivante.

IV. MES DEUX MISSIONS DE DÉVELOPPEMENT

Vers la deuxième moitié du stage, mon tuteur m'a confié deux missions de développement. C'est lui qui m'a fixé les objectifs et donné les indications de base, et c'est aussi lui, en tant que chef de l'équipe technique, qui décide à la fin si mon code est intégré au site ou non.

1. Mission 1 : widget des derniers articles

a) Le besoin exprimé

Mon tuteur m'a expliqué que la rédaction souhaitait mettre en avant les articles les plus récents dans une zone visible du site, en plus de la page d'accueil. L'idée était d'afficher dans la sidebar un petit bloc avec les 5 derniers articles publiés, leur titre, leur date, et un lien direct vers chacun d'eux.

Le cahier des charges était simple :

- Afficher les 5 derniers articles publiés, tous types confondus ;
- Pour chaque article : le titre cliquable, la date, et éventuellement la rubrique ;
- Le widget doit pouvoir être ajouté ou retiré facilement par l'admin ;
- Respecter le style visuel du thème existant.

b) Mon approche

WordPress propose une classe native pour créer ses propres widgets : `WP_Widget`. J'ai donc créé un fichier `widget-derniers-articles.php` que j'ai inclus depuis `functions.php`. Le widget utilise `WP_Query`, qui est la fonction recommandée par WordPress pour récupérer des articles, plutôt que d'écrire directement du SQL.

c) Le code

```
<?php
// Widget : Derniers articles AMP
class AMP_Derniers_Articles extends WP_Widget {

    public function __construct() {
        parent::__construct(
            'amp_derniers_articles',
            'AMP - Derniers articles',
            array('description' => 'Affiche les 5 derniers articles')
        );
    }

    public function widget($args, $instance) {
        echo $args['before_widget'];
    }
}
```

```

        echo '<h3>Derniers articles</h3>';

        $query = new WP_Query(array(
            'post_type'      => 'post',
            'post_status'    => 'publish',
            'posts_per_page' => 5,
            'orderby'        => 'date',
            'order'          => 'DESC'
        ));

        if ($query->have_posts()) {
            echo '<ul class="amp-derniers">';
            while ($query->have_posts()) {
                $query->the_post(); ?>

                <li>
                    <a href="<?php the_permalink(); ?>">
                        <?php the_title(); ?>
                    </a>
                    <span class="date">
                        <?php echo get_the_date(); ?>
                    </span>
                </li>

                <?php }
            echo '</ul>';
            wp_reset_postdata();
        }

        echo $args['after_widget'];
    }
}

// On enregistre le widget
function amp_register_widget() {
    register_widget('AMP_Derniers_Articles');
}
add_action('widgets_init', 'amp_register_widget');
```

d) Ce que j'ai appris

Cette mission, qui paraît simple, m'a fait toucher pas mal de choses : la programmation orientée objet en PHP, les hooks WordPress (add_action), et la logique de WP_Query qui m'évite d'écrire du SQL à la main. C'est mon tuteur qui m'a expliqué pourquoi il vaut mieux

passer par WP_Query plutôt que par une requête SQL directe : ça permet à WordPress d'appliquer ses propres filtres et son cache automatiquement.

2. Mission 2 : compteur de vues par article

a) Le besoin exprimé

La deuxième mission qu'on m'a confiée portait sur le suivi des vues. La rédaction voulait savoir, pour chaque article, combien de fois il avait été consulté. L'idée derrière, c'est de mesurer quels articles intéressent le plus les lecteurs.

Le cahier des charges :

- À chaque fois qu'un article est ouvert par un visiteur, son compteur augmente de 1 ;
- Le nombre de vues s'affiche en bas de chaque article;
- Le compteur ne doit pas s'incrémenter quand c'est un administrateur connecté qui ouvre l'article.

b) Mon approche

Pour stocker le compteur, j'ai utilisé le système de métadonnées d'articles que WordPress propose nativement (update_post_meta et get_post_meta). C'est plus simple et plus propre que créer une table à part, et c'est ce que recommande la documentation officielle. J'ai ensuite branché ma fonction sur le hook wp qui s'exécute au chargement de chaque page.

c) Le code

```
<?php
// À placer dans functions.php

// 1. Incrémente le compteur quand un article est consulté
function amp_compter_vue() {
    if (is_single() && !is_user_logged_in()) {
        $post_id = get_the_ID();
        $vues = get_post_meta($post_id, 'amp_vues', true);
        $vues = empty($vues) ? 0 : intval($vues);
        update_post_meta($post_id, 'amp_vues', $vues + 1);
    }
}

add_action('wp', 'amp_compter_vue');

// 2. Affiche le compteur en bas de l'article
function amp_afficher_vues($content) {
    if (is_single()) {
        $vues = get_post_meta(get_the_ID(), 'amp_vues', true);
        $vues = empty($vues) ? 0 : intval($vues);
```

```

    $content .= '<p class="amp-vues">
        Vu ' . $vues . ' fois</p>';
    }
    return $content;
}
add_filter('the_content', 'amp_afficher_vues');

```

d) Ce que j'ai appris

Cette mission m'a fait découvrir les métadonnées WordPress, qui sont une façon élégante d'attacher des infos à un article sans avoir à créer de table. J'ai aussi compris la différence entre les deux types de hooks que WordPress propose : les actions (add_action) qui font quelque chose à un moment précis, et les filtres (add_filter) qui modifient une donnée avant qu'elle soit affichée.

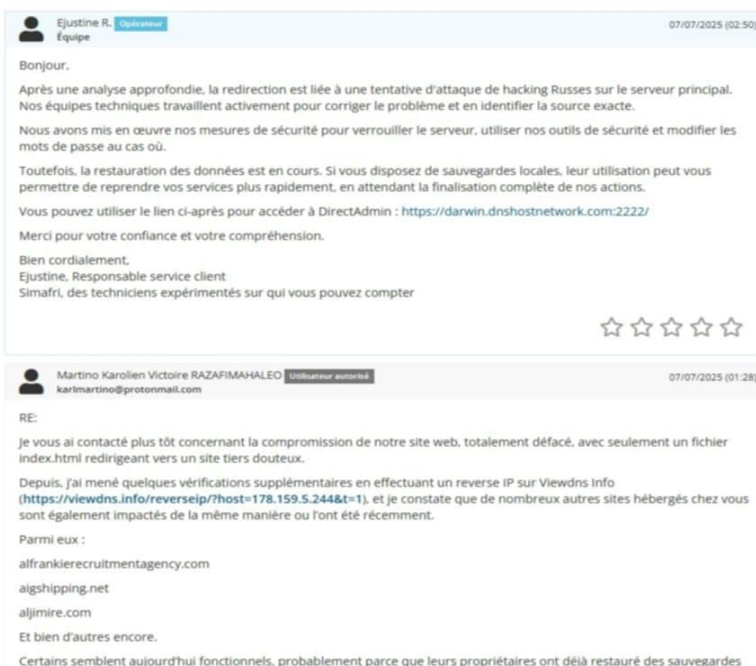
La petite subtilité, c'était de ne pas compter les vues des admins connectés. La fonction `is_user_logged_in()` m'a permis de gérer ça en une ligne.

V. LES INCIDENTS SERVEUR

1. L'attaque et le défacement

En juillet, le site a subi une tentative d'attaque. Le fichier `index.html` a été modifié et les visiteurs étaient redirigés ailleurs. Je n'étais pas en première ligne pour gérer ça, c'est l'administrateur principal et le service technique de l'hébergeur qui s'en sont occupés. Mais j'ai pu suivre la résolution de près.

Ce qu'ils ont fait : verrouiller le serveur, restaurer depuis les sauvegardes, changer tous les mots de passe, mettre à jour les composants. Le site est revenu en ligne en quelques heures.



Échange d'e-mails avec le service technique de l'hébergeur concernant la compromission du serveur et la tentative d'attaque externe.

2. Surcharges et coupures

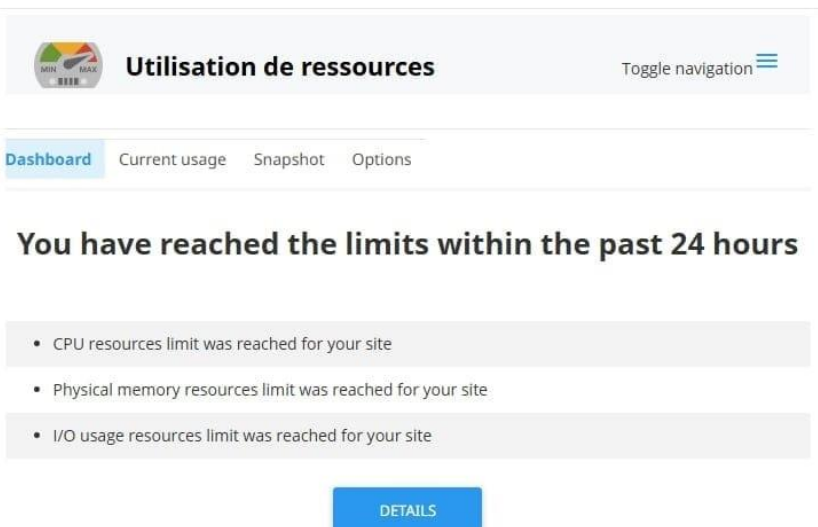
Plusieurs fois le tableau de bord a affiché des alertes de dépassement de ressources : surcharge CPU, mémoire pleine, saturation des entrées/sorties. À ces moments-là, le site ralentissait fortement ou tombait carrément.

Le module Jetpack Monitor envoyait aussi régulièrement des mails pour nous dire que le site était inaccessible. Certains jours on en recevait plusieurs.

Je n'ai pas résolu ces problèmes moi-même mais en les observant j'ai compris une chose : l'offre d'hébergement n'était plus adaptée au trafic réel du site. C'est un sujet qu'on n'aborde pas vraiment en cours, mais qui est très concret en entreprise. Il y a une vraie discussion budget à avoir entre le coût d'un hébergement plus solide et le niveau de stabilité que l'agence peut se payer.

Tableau de bord > Resource Usage

Resource Usage



Alerte de dépassement des ressources serveur

VI. BILAN : CE STAGE ET LE BTS SIO SLAM

1. Ce que j'ai vraiment appris

Premièrement, j'ai appris à apprendre vite, tout seul, dans un cadre pro. Découvrir WordPress en quelques jours alors que je n'y connaissais rien, c'est probablement mon plus gros acquis du stage. Cette capacité à se former rapidement sur un outil inconnu.

Deuxièmement, j'ai compris concrètement comment PHP, MySQL et le serveur web fonctionnent ensemble. Avant le stage c'était trois cours séparés dans ma tête. Maintenant c'est une chaîne dont je vois le fonctionnement global.

Et puis grâce aux deux missions que mon tuteur m'a confiées, j'ai mis le pied dans le développement WordPress concret : créer un widget avec WP_Widget, manipuler les hooks, utiliser WP_Query plutôt que d'écrire du SQL à la main, jouer avec les métadonnées d'articles. C'est mes premiers vrais bouts de PHP utiles.

Enfin, j'ai pu constater que la sécurité informatique n'est pas un sujet abstrait. Une attaque sur un site en production, c'est quelque chose qui arrive vraiment, et qui se traite avec méthode. Les notions étudiées en cours de prennent une autre dimension une fois confrontée à un cas réel.

2. Ce que je n'ai pas (encore) appris

Je veux aussi être honnête sur ce que je n'ai pas fait. Mes deux missions étaient des projets ciblés, pas une application complète. Je n'ai pas conçu de schéma de base de données, je n'ai pas mené un gros projet en autonomie de A à Z. Ces compétences, je devrai les acquérir lors de mon stage de deuxième année qui sera plus orienté développement.

Je n'ai pas non plus pratiqué le travail en équipe sur du code. C'est quelque chose que je commence à aborder en cours et que j'espère pratiquer en stage de fin d'études.

CONCLUSION

Ce stage à l'AMP a été ma première expérience d'un environnement informatique en production, ce que j'ai constaté :

La première : six semaines c'est court pour maîtriser un outil comme WordPress, mais c'est largement suffisant pour comprendre comment fonctionne un site dynamique de l'intérieur. J'ai vu PHP, MySQL et un serveur web travailler ensemble, j'ai développé mes deux premiers

vrais bouts de code WordPress, et j'ai assisté à un vrai incident de sécurité. Ce sont des choses qu'aucun cours ne peut totalement remplacer.

La deuxième : ce stage m'a confirmé que le choix de l'option SLAM était le bon. Lire et comprendre du code existant, réfléchir à comment il s'intègre dans une architecture, voir comment on peut l'améliorer.

Pour la suite, je sais ce qu'il me reste à apprendre : conduire un projet de développement en autonomie, maîtriser Git pour le travail en équipe, et approfondir la sécurité des applications web. J'espère pouvoir aborder ces points lors de mon stage de deuxième année.

Je termine en remerciant encore l'équipe de l'AMP, qui a accueilli un stagiaire débutant avec patience, et qui m'a donné une première image concrète du métier auquel je me destine.